VOID CTF – Writeups

Team: Code Crew

Authors:

- Arjun Singh Pundir (@ARJUN-PUNDIR) — Steganography, Reverse Engineering
- Akshat (@akshat-xcode) — Forensics

Categories: Steganography, Reverse Engineering, Forensics

CTF Challenge Writeup: The Vault

Challenge Name: The Vault

Points: <add points>

Challenge Description

We were given a single image, `challenge.jpg`, with the following story:

> "Our operatives intercepted a suspicious photograph from a rogue operative's terminal. Headquarters suspects there's more to this picture than meets the eye. They claim they left some sensitive files behind, but looking at the pixels alone won't save us. Can you dig beneath the surface and crack open The Vault?"

Flag format: `VOID{...}`

The hint "looking at the pixels alone won't save us" suggests the data is hidden *around* the image, not inside its pixels.

Step-by-Step Solution

Step 1: Check the file and its metadata

```
file challenge.jpg
```

`file` reported a JPEG with a **comment** field:

```
comment: "Qmkxa00zR0VoaHdjVlZ1blUyTHJjeWNMeE1QVkVnaVM3VDMyODZTQ0VCYkZoNWdkZE1LQg=="
```

Step 2: Decode the comment (a decoy)

The comment is Base64. Decoding it gives another string:

```
echo "Qmkxa00zR0VoaHdjVlZ1blUyTHJjeWNMeE1QVkVnaVM3VDMyODZTQ0VCYkZoNWdkZE1LQg==" | base64 -d
```

```
Bi1kM3GEhhwcVVunU2LrcycLxMPVEgiS7T3286SCEBbFh5gddMKB
```

This uses only Base58 characters, so we decoded it with Python:

```
A = '123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz'
s = "Bi1kM3GEhhwcVVunU2LrcycLxMPVEgiS7T3286SCEBbFh5gddMKB"
n = 0
for c in s:
    n = n * 58 + A.index(c)
print(n.to_bytes((n.bit_length() + 7) // 8, 'big'))
```

Output:

```
b'Haha... nice try. But this is a decoy.'
```

So the comment was a trap, and the real data is somewhere else.

Step 3: Look for data appended after the image

Running `strings` on the file showed file names near the end of it: `01_hidden.txt`, `02_hidden.txt`, `03_hidden.txt`. That means a **ZIP archive is appended after the JPEG data**. We located the ZIP signature (`PK\x03\x04`) and carved it out:

```
d = open('challenge.jpg', 'rb').read()
i = d.find(b'PK\x03\x04')
open('emb.zip', 'wb').write(d[i:])
```

```
unzip emb.zip
```

The archive was not password protected and contained three files.

Step 4: Identify the real file types

Although the files are named `.txt`, `file` showed they are images:

```
file 01_hidden.txt 02_hidden.txt 03_hidden.txt
```

File	Real type	Content
`01_hidden.txt`	JPEG	Desert dunes photo
`02_hidden.txt`	JPEG	Autumn waterfall photo
`03_hidden.txt`	PNG (1536x1024)	Wasteland scene with a power plant and a tattered flag

We renamed `03_hidden.txt` to `03.png` and opened it. The text on the flag in the picture is the flag.

<!-- add screenshot: 03.png -->

Flag

```
VOID{the_picture_was_inside_the_picture}
```

> Note: the text wraps over two lines on the banner (`VOID{the_picture_was` / `inside_the_picture}`). Join the lines with an underscore.

CTF Challenge Writeup: Afterimage

Challenge Name: Afterimage

Points: <add points>

Challenge Description

We were given `afterimage.png` , a night photograph of a fire and smoke over a city skyline, with this hint:

> "A photograph captured at the right moment can tell a story. Can you find what was left behind?"

Step-by-Step Solution

Step 1: Inspect the PNG chunks

We parsed the PNG chunk by chunk to look for metadata and data after `IEND` :

```
import struct
d = open('afterimage.png', 'rb').read()
i = 8
while i < len(d):
    l, t = struct.unpack('>I4s', d[i:i+8])
    print(t, l, d[i+8:i+8+l][:80] if t != b'IDAT' else '')
    i += 12 + l
    if t == b'IEND':
        break
print('trailing bytes:', len(d) - i)
```

There was no trailing data, but there were several text chunks (`iTXt` / `tEXt`) with a description, author, copyright and a **Comment**:

```
QC annotation (encoded): UUMtMTc6IEJMVUUgTFNCLCByb3ctbWFqb3I7IHZlcmlmeSBhcmNoaXZlIHJlZmVyZW5jZSBBRi0xNy4=
```

Step 2: Decode the QC annotation

```
echo "UUMtMTc6IEJMVUUgTFNCLCByb3ctbWFqb3I7IHZlcmlmeSBhcmNoaXZlIHJlZmVyZW5jZSBBRi0xNy4=" | base64 -d
```

```
QC-17: BLUE LSB, row-major; verify archive reference AF-17.
```

This tells us to read the **least significant bit of the blue channel**, in row-major order.

Step 3: Extract the blue LSB (a decoy again)

```
from PIL import Image
import numpy as np

im = np.array(Image.open('afterimage.png').convert('RGB'))
bits = (im[:, :, 2] & 1).flatten()
data = np.packbits(bits[:len(bits)//8*8].reshape(-1, 8), axis=1).flatten().tobytes()
print(data[:200].decode(errors='replace'))
```

Output:

```
ARCHIVE DECOY: no flag here.
AF-17 calibration record:
Horizontal drift: 1
Vertical drift: 3
Sampling cycle: 17
Residual target: 0
Active sensor: G
```

The blue channel is a decoy, but it leaves us a **calibration record** that describes where the real data is:

Field	Value	Meaning
Horizontal drift	1	x coefficient
Vertical drift	3	y coefficient
Sampling cycle	17	modulus
Residual target	0	remainder to match
Active sensor	G	use the **green** channel

Step 4: Sample the green channel using the calibration record

Only pixels where `(1*x + 3*y) mod 17 == 0` carry data. We took the green LSB at just those pixels, in row-major order:

```
from PIL import Image
import numpy as np

im = np.array(Image.open('afterimage.png').convert('RGB'))
H, W, _ = im.shape
ys, xs = np.mgrid[0:H, 0:W]

mask = ((1 * xs + 3 * ys) % 17 == 0)
g = im[:, :, 1] & 1
sel = g[mask]

n = len(sel) // 8 * 8
data = np.packbits(sel[:n].reshape(-1, 8), axis=1).flatten().tobytes()
print(data[:60])
```

Output (the bytes after the flag are random padding):

```
b'AIMG\x00(VOID{afterimage_pixels_are_the_evidence} ...'
```

The payload starts with an `AIMG` header followed by the flag.

Flag

```
VOID{afterimage_pixels_are_the_evidence}
```

CTF Challenge Writeup: QR Fragments

Challenge Name: <add challenge name>

Category: Steganography / Forensics

Points: <add points>

Challenge Description

We were given **12 image fragments** (`fragment_01` ... `fragment_12`). They are pieces of more than one picture, and not every signal in them is real.

Step-by-Step Solution

Step 1: Work out the layout

The 12 fragments assemble into **two separate 600x600 QR codes**. Each QR code is a **3x2 grid** (3 columns, 2 rows) of 200x300 fragments. Checking edge continuity and the padding around each tile shows which fragments belong together and in what order.

Step 2: Reassemble Signal 1

Layout (row by row, left to right):

	Col 1	Col 2	Col 3
Row 1	fragment_03	fragment_01	fragment_09
Row 2	fragment_02	fragment_06	fragment_11

Scanning the reassembled QR code gives:

```
VOID{s33_th3_wh0l3_p1ctur3}
```

Step 3: Reassemble Signal 2

Layout:

	Col 1	Col 2	Col 3
Row 1	fragment_04	fragment_07	fragment_05
Row 2	fragment_10	fragment_12	fragment_08

Scanning this QR code gives:

```
VOID{n0t_3v3ry_s1gn4l_1s_r34l}
```

Step 4: Separate the real signal from the decoy

Signal 2 reads "not every signal is real", so it announces that it is the decoy transmission. Signal 1 reads "see the whole picture", which is what you get by putting all the fragments back together correctly. Signal 1 is the real flag.

Flag

```
VOID{s33_th3_wh0l3_p1ctur3}
```

CTF Challenge Writeup: chall (LoongArch Reverse Engineering)

Challenge Name: chall

Category: Reverse Engineering

Points: <add points>

Challenge Description

We were given a 64-bit **LoongArch ELF binary**, `chall`, and the output it produces for the correct 30-character input:

```
-655162063 -838009797 232397087 -1738908707 -468405423 -143659455 1211474117 1874553211 -143659455 -489127
```

The goal is to recover the input, which is the flag.

Step-by-Step Solution

Step 1: Reverse the binary

Reversing the binary shows how an input of length 30 is transformed. `arr` below is the array of the input's character codes.

`func_8bc` (left = 0, right = 29, len = 30)

- Indices 0 to 3: `arr[i] = arr[i] - 299`
- Index 4: unchanged
- Indices 5 to 29: `arr[j] = arr[j] * 67`
- It then splits the array at `mid = 15`, calling `func_a80(arr, 0, 14)` on the first half and `func_c44(arr, 15, 29)` on the second half.

First half, `func_a80` (indices 0 to 14)

```
arr[k] = (arr[k] * 0x9e3779b1 + 123) mod 2^32
```

Second half, `func_c44` (indices 15 to 29)

```
arr[j] = 17 * ((arr[j] ^ 912) ^ 91) = 17 * (arr[j] ^ 971)
```

Final step in `main` (for all i from 0 to 29)

```
v2        = (6 * (arr[i] + 7)) mod 2^32
output[i] = (~v2) + 100000
```

Step 2: Invert the output step

From `output` back to `arr`:

```
v2 = ~(output[i] - 100000)  (mod 2^32)
```

`v2` is even, so `v2 / 2 = 3 * (arr[i] + 7) (mod 2^31)`. We invert this modulo 2^31 (this leaves two candidates modulo 2^32).

Step 3: Invert the first half (indices 0 to 14)

- Undo the affine step using the modular inverse `0x9e3779b1^-1 = 0x0e8b2f51 (mod 2^32)`.
- Add 299 back for indices 0 to 3.
- Divide by 67 for indices 5 to 14.

This gives:

```
VOID{1_l1k3_ic3
```

Step 4: Invert the second half (indices 15 to 29)

- Divide by 17.
- XOR with 971.
- Divide by 67.

This gives:

```
_cr34m_t00!!!!}
```

Step 5: Check the result

Running the forward algorithm on the recovered string reproduces all 30 output values exactly.

Flag

```
VOID{1_l1k3_ic3_cr34m_t00!!!!}
```

Forensics Challenges

CTF Challenge Writeup: I Heard something

Team: Code Crew

Author: Akshat

Category: Forensics

Points: 100

Flag: `VOID{y0u_c4n7_h34r_m3}`

Challenge Description

> "Mira left a voice note on her desktop called 'whisper.' It's just a tune. Or is it? Some things are only meant for machines to hear."

We were provided with an audio file (`whisper.wav`).

Flag format: `VOID{...}`

Technical Properties

Property	Value
File Format	Microsoft WAV (RIFF `WAVE` , uncompressed PCM)
File Size	1,058,444 bytes
Audio Channels	1 (Mono)
Bit Depth	16-bit signed PCM (`int16` , little-endian)
Sampling Rate	44,100 Hz (Nyquist limit: 22,050 Hz)
Duration	12.000 seconds (529,200 samples)

Step-by-Step Solution

Step 1: Initial Container Inspection & Metadata Audit

We inspected the RIFF chunk headers and tested whether any extra data was appended after the `data` chunk:

```
file whisper.wav
exiftool whisper.wav
```

The RIFF container matched its declared file size exactly, and no suspicious metadata tags or appended payload strings were found in the header chunks.

Step 2: Time-Domain & LSB Steganography Check

We inspected the raw PCM sample values for standard least-significant-bit (LSB) steganographic encoding. Extracting the LSBs across byte alignments showed typical uniform noise distribution and natural audio entropy, with no file signatures or text strings.

Step 3: Frequency-Domain (Spectrogram) Analysis

The hint stated:

> *"Some things are only meant for machines to hear."*

Human hearing generally extends up to ~15–17 kHz for adults, while digital audio sampled at 44.1 kHz can represent frequencies up to the Nyquist limit of 22,050 Hz.

We performed a Short-Time Fourier Transform (STFT) across the entire spectrum (0 Hz to 22.05 kHz). While the audible acoustic tune was concentrated in the low-frequency region ($0 - 1.5\text{ kHz}$), a clear synthetic spectral drawing appeared in the **high-frequency ultrasonic band between 16.3 kHz and 19.3 kHz**:

- **Start ($t \approx 2.05\text{ s}$):** `VOID{`
- **Body ($t \approx 3.0\text{ s} - 9.0\text{ s}$):** Visual glyphs spelling `y0u_c4n7_h34r_m3` (with distinct horizontal bar strokes for underscores `_` at ~16.4 kHz)
- **End ($t \approx 9.9\text{ s}$):** `}`

Reproducible Python Script

```
import numpy as np
import scipy.io.wavfile as wavfile
import matplotlib.pyplot as plt

# 1. Load the challenge audio file
sr, data = wavfile.read("whisper.wav")

# 2. Generate high-frequency zoomed spectrogram
plt.figure(figsize=(14, 6))
Pxx, freqs, bins, im = plt.specgram(
    data,
    NFFT=2048,
    Fs=sr,
    noverlap=1024,
    cmap='viridis'
)

# Focus on the ultrasonic band
plt.ylim(15000, 21000)
plt.title("High-Frequency Spectrogram (15 kHz - 21 kHz)")
plt.xlabel("Time (seconds)")
plt.ylabel("Frequency (Hz)")
plt.colorbar(im, label="Intensity (dB)")
plt.tight_layout()
plt.savefig("flag_spectrogram.png", dpi=300)
plt.show()
```

Opening the resulting spectrogram clearly displays:

```
VOID{y0u_c4n7_h34r_m3}
```

CTF Challenge Writeup: Left Behind

Team: Code Crew

Author: Akshat

Category: Forensics

Points: 200

Flag: `VOID{h1d_k3ystr0k3s}`

Challenge Description

> "Arlo Vance runs the server room at Halvorsen Logistics and trusts nothing that touches a network. His workstation is air-gapped, and his passwords live only in his head. During last night's audit, someone left a small device plugged in behind his desk. It sat there quietly until Arlo noticed it this morning and yanked it out. This file is all it left behind. Arlo insists there was nothing on it worth worrying about. Prove him wrong."

We were provided with a network capture file: `left_behind.pcapng`.

Flag format: `VOID{...}`

File Identification & Forensics

Property	Value
Filename	`left_behind.pcapng`
File Size	110,524 bytes
File Format	PCAP Next Generation Capture File (`pcapng`)
Link Layer Type	`LINKTYPE_USB_LINUX_MMAPPED` (Linktype 220)
Capture Origin	Linux kernel `usbmon` subsystem
Total Packets	1,100 Enhanced Packet Blocks (EPBs)
SHA-256	`e6ebca6440909a849acd6ed3889b22064b7ef7c3dc17682880bf38a654656e07`

Step-by-Step Solution

Step 1: Identifying the Hardware Sniffer Traffic

Opening the capture file reveals it is not an Ethernet/IP capture, but a raw USB capture recorded using Linux's `usbmon` facility (Linktype 220).

Inspecting the USB descriptors in the initial setup/control transfers identified 4 active devices on the USB bus:

1. **Device 1:** Linux Foundation Root Hub (`VID 0x1d6b, PID 0x0002`)

2. **Device 2:** Intel Corp. Integrated Hub (`VID 0x8087, PID 0x0024`)

3. **Device 3:** Primax Electronics Optical Mouse (`VID 0x0461, PID 0x4d0f`) — 466 packets on EP `0x81`

4. **Device 4:** Logitech USB Keyboard (`VID 0x046d, PID 0xc31c`) — 72 interrupt packets on EP `0x81`

The small device plugged behind Arlo's desk was an inline hardware keylogger capturing USB traffic from **Device 4**.

Step 2: Decoding USB HID Keyboard Reports

Device 4 sends standard 8-byte USB HID Keyboard Boot Protocol reports:

- **Byte 0:** Modifier bitmask (`0x02` = Left Shift, `0x20` = Right Shift, `0x04` = Left Alt)
- **Byte 1:** Reserved (`0x00`)
- **Bytes 2–7:** Up to 6 active key usage IDs (from the USB HID Usage Tables specification)

Keycodes of interest:

- `0x04` - `0x1d` : Letters `a` - `z` (or `A` - `Z` if Shift is held)
- `0x1e` - `0x27` : Numbers `1` - `0`
- `0x2a` : Backspace (`\b`)
- `0x2d` : `-` / `_`
- `0x2f` : `[` / `{`
- `0x30` : `]` / `}`

Step 3: Handling Typo & Backspaces

Analyzing the raw keystroke sequence:

1. `VOID{h1d_` is typed.
2. The user types `wrong` (5 characters).
3. The user presses **Backspace 5 times** (`0x2a`), erasing `wrong` .
4. The user completes typing: `k3ystr0k3s}` .

Final reconstructed string:

```
VOID{h1d_k3ystr0k3s}
```

Reproducible Python Script

```
import struct

PCAP_PATH = "left_behind.pcapng"

# Standard USB HID Keyboard Mapping
HID_MAP = {
    0x04: ('a', 'A'), 0x05: ('b', 'B'), 0x06: ('c', 'C'), 0x07: ('d', 'D'),
    0x08: ('e', 'E'), 0x09: ('f', 'F'), 0x0a: ('g', 'G'), 0x0b: ('h', 'H'),
    0x0c: ('i', 'I'), 0x0d: ('j', 'J'), 0x0e: ('k', 'K'), 0x0f: ('l', 'L'),
    0x10: ('m', 'M'), 0x11: ('n', 'N'), 0x12: ('o', 'O'), 0x13: ('p', 'P'),
    0x14: ('q', 'Q'), 0x15: ('r', 'R'), 0x16: ('s', 'S'), 0x17: ('t', 'T'),
    0x18: ('u', 'U'), 0x19: ('v', 'V'), 0x1a: ('w', 'W'), 0x1b: ('x', 'X'),
    0x1c: ('y', 'Y'), 0x1d: ('z', 'Z'), 0x1e: ('1', '!'), 0x1f: ('2', '@'),
    0x20: ('3', '#'), 0x21: ('4', '$'), 0x22: ('5', '%'), 0x23: ('6', '^'),
    0x24: ('7', '&'), 0x25: ('8', '*'), 0x26: ('9', '('), 0x27: ('0', ')'),
    0x2a: ('\b', '\b'), # Backspace
    0x2d: ('-', '_'),   0x2e: ('=', '+'), 0x2f: ('[', '{'), 0x30: (']', '}'),
    0x2b: ('\t', '\t'), 0x2c: (' ', ' ')
}

with open(PCAP_PATH, "rb") as f:
    raw = f.read()

offset = 0
prev_keys = set()
buffer = []

while offset < len(raw):
    bt, bl = struct.unpack("<II", raw[offset:offset+8])
    body = raw[offset+8 : offset+bl-4]

    # Enhanced Packet Block (EPB)
    if bt == 6:
        cap_len = struct.unpack("<I", body[12:16])[0]
        pkt = body[20:20+cap_len]

        # 64-byte Linux usbmon_packet header + payload
        if len(pkt) >= 72:
            urb_type = chr(pkt[8])
            epnum = pkt[10]
            devnum = pkt[11]
            payload = pkt[64:]

            # Device 4 = Keyboard, Endpoint 0x81 (IN interrupt transfer)
            if devnum == 4 and epnum == 0x81 and urb_type == 'C':
                mod = payload[0]
                shift = bool(mod & 0x22)
```

```
                curr_keys = set(k for k in payload[2:8] if k != 0)
                new_keys = curr_keys - prev_keys

                for k in new_keys:
                    if k in HID_MAP:
                        char = HID_MAP[k][1] if shift else HID_MAP[k][0]
                        if char == '\b':
                            if buffer:
                                buffer.pop()
                        elif not (mod & 0x44): # Ignore system Alt combos
                            buffer.append(char)
                prev_keys = curr_keys

    offset += bl

print(f"[+] Recovered Flag: {''.join(buffer)}")
```

Output:

```
[+] Recovered Flag: VOID{h1d_k3ystr0k3s}
```

CTF Challenge Writeup: A Sealed Record

Team: Code Crew

Author: Akshat

Category: Forensics

Points: 300

Flag: `VOID{crc_r3v34ls_th3_tru3_h31ght}`

Challenge Description

> "An evidence image came in from intake, but something's off — a few tools refuse to open it at all. The analyst who logged it left one note: 'the tag was rewritten after sealing. the seal still remembers the original. trust the seal, not the tag.' Recover the record."

We were provided with an archive: `Artifact-BC34.rar`.

Flag format: `VOID{...}`

File Identification & Forensics

Outer Archive (`Artifact-BC34.rar`)

* **Format:** RAR 5.0 Archive

* **Signature:** `52 61 72 21 1A 07 01 00` (`Rar!\x1a\x07\x01\x00`)

* **SHA-256:** `9267c6daad76ec2da7849c7d70ff0069e520df5ce260b039cb92c6591fa6fff5`

* **Contained File:** `Artifact-BC34.png`

Evidence Image (`Artifact-BC34.png`)

* **Format:** Portable Network Graphics (PNG)

* **Signature:** `89 50 4E 47 0D 0A 1A 0A`

* **File Size:** 6,343,346 bytes

* **Tampered Dimensions:** $2464 \times 1607\text{ px}$

* **Original Dimensions:** $2464 \times 1728\text{ px}$

Step-by-Step Solution

Step 1: Analyzing the Analyst's Clue

The intake note says:

> *"the tag was rewritten after sealing. the seal still remembers the original. trust the seal, not the tag."*

In the PNG format:

- Each chunk has a **4-byte chunk type** (e.g. `IHDR`) and chunk data.
- Following the chunk data is a **4-byte CRC-32 checksum** ("the seal") calculated over chunk type + data.

- The `IHDR` chunk contains width (4 bytes), height (4 bytes), bit depth, color type, compression, filter, and interlace methods.

When strict image viewers or forensic parsers attempted to render `Artifact-BC34.png`, they threw a CRC mismatch error on the `IHDR` chunk because the height field had been modified without recalculating the CRC.

Step 2: Comparing Stored CRC vs Calculated CRC

Examining the `IHDR` chunk in a hex editor:

- Width: `00 00 09 A0` (2464 px)
- Stored Height: `00 00 06 47` (1607 px)
- Other fields: `08 02 00 00 00` (8-bit Truecolor)
- Stored CRC-32 checksum: `0x14C4F49F`

Calculating the CRC-32 of `IHDR` with height 1607 px gives `0x1771D0FF`, which **does not match** `0x14C4F49F`.

Step 3: Brute-Forcing the Original Height via CRC-32

Because "the seal still remembers the original", we brute-forced the height parameter against the valid stored CRC `0x14C4F49F`:

```
import struct, zlib

width = 2464
stored_crc = 0x14C4F49F

for h in range(1, 10000):
    ihdr_payload = b'IHDR' + struct.pack('>IIBBBBB', width, h, 8, 2, 0, 0, 0)
    if zlib.crc32(ihdr_payload) == stored_crc:
        print(f"[+] True original height: {h} (0x{h:08X})")
        break
```

Output:

```
[+] True original height: 1728 (0x000006C0)
```

The image height had been artificially truncated by **121 vertical pixels** ($1728 - 1607 = 121$).

Step 4: Repairing the PNG & Revealing the Flag

We patched bytes `0x14 - 0x17` from `00 00 06 47` to `00 00 06 C0`.

Opening the restored image reveals the previously hidden 121 bottom rows displaying an intake evidence terminal banner:

```
recovered record:
VOID{crc_r3v34ls_th3_tru3_h31ght}
```

Reproducible Python Script

```
import struct
import zlib

png_file = "Artifact-BC34.png"
repaired_file = "Artifact-BC34-repaired.png"

with open(png_file, "rb") as f:
    data = bytearray(f.read())

assert data[:8] == b"\x89PNG\r\n\x1a\n", "Invalid PNG header"

# IHDR fields
width = struct.unpack(">I", data[16:20])[0]
tampered_h = struct.unpack(">I", data[20:24])[0]
stored_crc = struct.unpack(">I", data[29:33])[0]

print(f"[+] Current Dimensions: {width} x {tampered_h}")
print(f"[+] Stored CRC: {hex(stored_crc)}")

# Recover original height
true_h = None
for candidate_h in range(1, 10000):
    ihdr_payload = b'IHDR' + struct.pack(">IIBBBBB", width, candidate_h, 8, 2, 0, 0, 0)
    if zlib.crc32(ihdr_payload) == stored_crc:
        true_h = candidate_h
        break

if true_h:
    print(f"[+] Found True Height: {true_h}")
    # Patch height in binary data
    data[20:24] = struct.pack(">I", true_h)
    with open(repaired_file, "wb") as f:
        f.write(data)
    print(f"[+] Saved repaired image to {repaired_file}")
else:
    print("[-] Height recovery failed")
```

CTF Challenge Writeup: Volatile Mira

Team: Code Crew

Author: Akshat

Category: Forensics

Points: 300

Flag: `VOID{f0r3n51c5_15_4n_4r7}`

Challenge Description

> "Mira never writes secrets to disk. 'Disk is evidence,' she wrote in her diary. 'RAM forgets.' Her notes daemon keeps everything encrypted in memory, and she swears the key never sits next to the vault. When her laptop went dark, our responder didn't pull the plug. She grabbed a full image of the machine's RAM first. Mira was wrong about one thing: RAM forgets only when the power goes. Find what she was hiding."

We were provided with a compressed memory dump archive: `mem.rar`.

Flag format: `VOID{...}`

Memory Artifact Identification

Property	Value
Archive	`mem.rar` (78,396,448 bytes)
Extracted Image	`mem.lime` (2,147,019,840 bytes / 2.0 GB)
Acquisition Format	**LiME (Linux Memory Extractor)** format v1
Magic Header	`45 4d 69 4c` (`0x4C694D45` / `EMiL` little-endian)
Target OS / Host	Kali GNU/Linux (x86_64)
SHA-256 (`mem.rar`)	`ed1e486b35e2d0c0c6b85549e3c7962e5d6f4ffc5e2924dcee542e8ff881db89`

Step-by-Step Solution

Step 1: Identifying the In-Memory Notes Daemon

We scanned the memory image for references to `mira`, `vault`, and python script buffers.

At physical offset `0x301E8FDB`, we recovered the Python source code of Mira's memory resident daemon (`/opt/mira/heartbeat.py`):

```
# Mira's "encrypted notes sync" daemon. Holds the XOR-encrypted flag in memory.
# Only ciphertext lives here: the plaintext flag never exists in the VM.
import time
SEALED = bytes.fromhex("632c7f761e530146035b020402566804003c025c3a0143034d")
# The marker makes the blob findable in the process dump.
# Hard mode: remove the marker so solvers must XOR-scan the dump for "VOID{".
vault = bytearray(b"MIRA-VAULT::" + SEALED + b"::END")
while True:
    time.sleep(60)  # stay alive so the process is in pslist at capture time
```

Ciphertext recovered:

`SEALED = bytes.fromhex("632c7f761e530146035b020402566804003c025c3a0143034d")` (25 bytes)

Step 2: Locating the Key ("The key never sits next to the vault")

The prompt noted:

> *"she swears the key never sits next to the vault."*

Since the key was not hardcoded in the script or in the vault array, it had to be supplied from outside at runtime.

We searched memory for process argument vectors (`mm_struct->arg_start`), bash command histories, and `/proc/[pid]/cmdline` allocations.

At offsets `0x13FFD4B6` and `0x15978CDD`, we discovered the exact execution command:

```
./launch.sh 5c62e5140575ac75
shred -u launch.sh
python3 /opt/mira/heartbeat.py --session-token 5c62e5140575ac75 --interval 60
```

The encryption key is the session token:

```
Key: 5c62e5140575ac75 (16 ASCII bytes)
```

Step 3: Decrypting the Sealed Vault

The encryption scheme is repeating-key XOR ($P_i = C_i \oplus K_{i \pmod{16}}$).

Byte-by-Byte Decryption Trace:

Index i	Cipher Byte C_i	Key Byte $K_{i \pmod{16}}$	Plaintext Char P_i
0	`0x63`	`'5'` (`0x35`)	**V**
1	`0x2c`	`'c'` (`0x63`)	**O**
2	`0x7f`	`'6'` (`0x36`)	**I**
3	`0x76`	`'2'` (`0x32`)	**D**
4	`0x1e`	`'e'` (`0x65`)	**{**
5	`0x53`	`'5'` (`0x35`)	**f**
6	`0x01`	`'1'` (`0x31`)	**0**
7	`0x46`	`'4'` (`0x34`)	**r**
8	`0x03`	`'0'` (`0x30`)	**3**
9	`0x5b`	`'5'` (`0x35`)	**n**
10	`0x02`	`'7'` (`0x37`)	**5**
11	`0x04`	`'5'` (`0x35`)	**1**
12	`0x02`	`'a'` (`0x61`)	**c**
13	`0x56`	`'c'` (`0x63`)	**5**
14	`0x68`	`'7'` (`0x37`)	_
15	`0x04`	`'5'` (`0x35`)	**1**
16	`0x00`	`'5'` (`0x35`)	**5**
17	`0x3c`	`'c'` (`0x63`)	_
18	`0x02`	`'6'` (`0x36`)	**4**
19	`0x5c`	`'2'` (`0x32`)	**n**
20	`0x3a`	`'e'` (`0x65`)	_
21	`0x01`	`'5'` (`0x35`)	**4**

Index i	Cipher Byte C_i	Key Byte $K_{i \pmod{16}}$	Plaintext Char P_i
22	`0x43`	`'1'` (`0x31`)	**r**
23	`0x03`	`'4'` (`0x34`)	**7**
24	`0x4d`	`'0'` (`0x30`)	**}**

Reproducible Python Script

```
SEALED = bytes.fromhex("632c7f761e530146035b020402566804003c025c3a0143034d")
KEY = b"5c62e5140575ac75"

flag = bytes([c ^ KEY[i % len(KEY)] for i, c in enumerate(SEALED)])
print(f"[+] Recovered Flag: {flag.decode('utf-8')}")
```

Output:

```
[+] Recovered Flag: VOID{f0r3n51c5_15_4n_4r7}
```

CTF Challenge Writeup: 3000 Files Later

Team: Code Crew

Author: Akshat

Category: Forensics

Points: 500

Flag: `VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}`

Challenge Description

> "Someone poured their whole life into a zip file. Thousands of flags, every one of them convincing. They swear only one is real, and that they left a way to prove it. Note - Before you start grepping: they saw you coming."

We were provided with a ZIP archive: `3000_files_later.zip`.

Flag format: `VOID{...}`

File Identification & Forensics

Property	Value
Filename	`3000_files_later.zip`
Archive Size	693,520 bytes
Total Files	3,369 files (across 256 subdirectories)
File Contents	Base64-encoded strings matching `VOID{...}`
SHA-256	`6e9f1502d2253d1d4c6e7569dbfa89229f7b05ca04816e0cf6ebeb481fa760eb`

Step-by-Step Solution

Step 1: Examining the Decoy Minefield

Extracting the archive reveals 3,369 text files, all containing Base64 strings. Decoding any file produces a convincing flag candidate, e.g.:

- `VOID{m3m0ry_l34k_f1x_d7a8e291}`
- `VOID{c4ch3_m1ss_3rr0r_5f2b8a1c}`

Grepping for `VOID{` yields over 3,000 matches. As the challenge author noted: *"they saw you coming."* There must be a mathematical or metadata verification mechanism that differentiates the authentic flag from the decoys.

Step 2: The Archive Comment Clue

We inspected the ZIP archive header comments:

```
import zipfile
zf = zipfile.ZipFile("3000_files_later.zip")
print(zf.comment.decode())
```

Output:

> `Ask the one that marks the beginning of an era how to sign. Trust the youngest who bears the mark.`

Step 3: "The one that marks the beginning of an era"

In the MS-DOS/FAT timestamp format used by ZIP files, the earliest representable timestamp (the epoch) is:

```
1980-01-01 00:00:00
```

Scanning all 3,369 entries for the minimum timestamp finds exactly one file:

- **Path:** `ae/4a/d9dc427dacb85135.txt`
- **Timestamp:** `1980-01-01 00:00:00`
- **Decoded Content:** `VOID{my_t41l_th3n_y0ur_b0dy_sha256_f1rst_8_1s_y0ur_t41l_1e7ea419}`

This file reveals the signing rule:

1. `my_t41l` = `1e7ea419`

2. Each candidate flag has structure: `VOID{<body_without_tail>_<tail>}`

3. The valid signature formula:

$$\text{Expected Tail} = \operatorname{SHA256}(\mathtt{"1e7ea419"} \,\|\, \text{body})[0:8]$$

Step 4: Filtering Candidate Flags

We evaluated all 3,369 files against this SHA-256 signature verification rule.

Exactly **8 files** in the entire archive "bear the mark" (valid signature):

Modification Timestamp	File Path	Flag Content
2004-03-07 21:51:16	`5a/3b/b517d9196b0521aa.txt`	`VOID{0ld_sch00l_fl4g_s1gn_d2c88f11}`
2006-11-12 14:20:02	`1f/c2/a7e3d1c8b9a04f21.txt`	`VOID{b1t_sh1ft_m4st3r_c4b910e5}`
2008-08-19 09:15:44	`8c/41/d0298be5f123ac78.txt`	`VOID{st4ck_0v3rfl0w_c0ur13r_fa22d861}`
2010-01-25 18:42:10	`3d/7a/e9821bf40c1157d3.txt`	`VOID{k3rn3l_p4n1c_h4ndl3r_88e04b12}`
2011-06-30 04:05:58	`92/e5/f83726ab5c010a34.txt`	`VOID{crypt0_k3y_r0t4t10n_9cb72e04}`
2013-09-14 22:11:30	`d4/12/38e9b0cd47f128bc.txt`	`VOID{s3cur3_v4ult_pr0t0c0l_374fa90c}`
2014-10-03 11:55:18	`0e/88/47a29dc5e01f6542.txt`	`VOID{p4ck3t_dr0p_4n4lys1s_4a19cb7d}`
2015-02-04 06:30:32	`b1/ab/54b611bcdb191433.txt`	`VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}`

Step 5: "Trust the youngest who bears the mark"

The youngest (most recently timestamped) file among the 8 signed candidates is:

- **Timestamp:** `2015-02-04 06:30:32`
- **File:** `b1/ab/54b611bcdb191433.txt`
- **Body:** `y0u_ch3ck3d_3000_f1l3s_f0r_th1s`
- **Verification:**

$$\operatorname{SHA256}(\mathtt{"1e7ea419y0u_ch3ck3d_3000_f1l3s_f0r_th1s"}) = \mathtt{\mathbf{021959f6}cebd...}$$

- **Tail:** `021959f6` (Exact match)

Reproducible Python Script

```
import zipfile
import base64
import hashlib
import datetime

ZIP_PATH = "3000_files_later.zip"

with zipfile.ZipFile(ZIP_PATH, 'r') as zf:
    # 1. Print comment
    print(f"[+] ZIP Comment: {zf.comment.decode('utf-8')}")

    # 2. Find beginning of era (DOS epoch 1980-01-01)
    epoch_entry = min(zf.infolist(), key=lambda x: datetime.datetime(*x.date_time))
    epoch_content = base64.b64decode(zf.read(epoch_entry.filename).strip()).decode('utf-8')
    print(f"[+] Epoch File: {epoch_entry.filename} ({datetime.datetime(*epoch_entry.date_time)})")
    print(f"[+] Rule Template: {epoch_content}")

    my_tail = epoch_content[5:-1].rsplit("_", 1)[1]

    # 3. Find all files bearing the mark
    signed_candidates = []
    for info in zf.infolist():
        try:
            content = base64.b64decode(zf.read(info.filename).strip()).decode('utf-8')
        except Exception:
            continue

        if content.startswith("VOID{") and content.endswith("}"):
            inner = content[5:-1]
            if "_" in inner:
                body, tail = inner.rsplit("_", 1)
                if len(tail) == 8:
                    expected_tail = hashlib.sha256((my_tail + body).encode()).hexdigest()[:8]
                    if expected_tail.lower() == tail.lower():
                        dt = datetime.datetime(*info.date_time)
                        signed_candidates.append((dt, info.filename, content))

    # 4. Sort by timestamp descending to get the youngest
    signed_candidates.sort(key=lambda x: x[0])
    youngest = signed_candidates[-1]

    print("\n[+] All 8 Signed Files:")
    for dt, fn, flg in signed_candidates:
        print(f"  [{dt}] {fn} -> {flg}")

    print(f"\n[+] Authentic Flag (Youngest): {youngest[2]}")
```

Output:

```
[+] Authentic Flag (Youngest): VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}
```
